

Using Time-of-Flight Sensors for People Counting Applications

Michal Stec, Viktor Herrmann
Embedded Systems Group
Fraunhofer Heinrich Hertz Institute, HHI
Berlin, Germany
michal.stec@hhi.fraunhofer.de
viktor.herrmann@hhi.fraunhofer.de

Benno Stabernack
Embedded Systems Group
Fraunhofer Heinrich Hertz Institute, HHI
Berlin, Germany
Embedded Systems Architectures for Signal Processing
University of Potsdam
Potsdam, Germany
benno.stabernack@hhi.fraunhofer.de

Abstract—Precisely detecting and counting people who are using public transportation is one of the key methods for predicting and planning an efficient use of buses, trams and trains. Providing an effective, well-planned public transportation service is not only important for economic reasons. It also helps to tackle a variety of environmental problems and contributes to a reduction of traffic congestion in urban areas. A couple of such systems had been developed in the past. Those were not sufficiently precise, however. In most cases, these systems rely on data processing generated by one particular type of a 2D image sensor. In this paper we present a robust people counting application, which runs on embedded systems with reasonable requirements as far as computational power is concerned and relies on the processing of 3D data generated by a Time-of-Flight (ToF) sensor. Processing of time-of-flight data requires a couple of preprocessing steps, which is crucial for the subsequent people detection, tracking and counting algorithms. The influence of these preprocessing steps and the effect on the developed detection algorithm are presented. Methods of avoiding misinterpretations by the detection algorithms are discussed. A detailed description of the core algorithms which were developed to process 3D data is provided. An overview will be given on how this method could be further enhanced for the purpose of detecting and differentiating vital and non-vital objects.

Index Terms—Time-of-Flight, 3D Vision, People Detection, People Counting, Public Transportation,

I. INTRODUCTION

Many people counting algorithms have already been described in scientific literature. Although they have essentially the same objective, they differ substantially in terms of the technology used, setup, boundary conditions, as well as their particular application.

One of the most simple methods was proposed in [8]. It utilizes only one IP camera, positioned above the door and registering the scene from above. After modeling the background and applying a few morphological operations to the foreground blobs, persons are detected and tracked. The algorithm performed well as long as only one person at a time had to be tracked and counted.

This strong restriction makes the aforementioned design unsuitable for more demanding applications. Moreover, using

The work has been funded by the German Federal Ministry of Economic Affairs and Energy (BMWi), project name: MSF Sensing

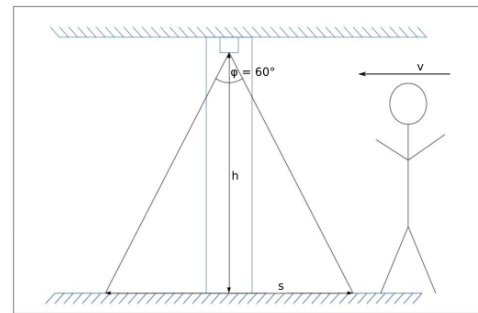


Fig. 1: Stickman getting counted

an IP camera as a single data source may not be sufficient if the lighting conditions were poor.

Another people counting algorithm based on just one camera was described in [4]. The objective was to count passengers at a railway station, hence under stable lighting. A camera had been placed at a considerable height above people's heads, looking down perpendicularly towards the floor. After background subtraction, each person's head can be detected using the Hough circle method on an image calculated by Canny's edges detector. The Horn-Schunck technique is used to track objects in consecutive frames. Before a person gets counted, their trajectory has to undergo spatio-temporal validation based on approximate median background subtraction. This last step prevents different parts of the body or other items to be counted as separate objects.

More advanced applications often make use of 3D information about the viewed area. There are basically two methods of generating a spacial model of a scene. The first utilizes a pair of cameras and, as a consequence of different viewing angles, the disparity image. A practical application of this approach can be found in [3], where a stereo vision system was incorporated to count passengers who are using public transportation. Cameras are placed above doors, but this time they are facing the floor at an angle. The algorithm tracks

objects that have a height within a specific range and are present in a defined region of interest. Targets are projected onto an occupancy grid which helps to resolve simple occlusions. The interframe association between blobs uses a feature-based similarity measure.

The second method uses a 3D model generated by a time-of-flight camera. One of the very first people tracking algorithms based on this technology was described in [6]. The setup consisted of a down-looking time-of-flight camera, placed 3 m above the ground.

Since Microsoft's Kinect was released, the number of applications using multi-sensor data fusion increased significantly. In [5] this very device is utilized to count people moving in the scene. The authors put the Kinect sensor top-down just above the door, at a height of approximately 2.4m. In order to be able to track people with a height of up to 2m, and due to the visual range of Kinect sensor, an additional lens had to be used.

With no constraints on the sensor's position, another algorithm using RGB-D data is proposed in [7]. The authors benefit from a larger scene depth with a camera that wasn't looking top-down anymore, but placed at an angle. After the foreground is extracted, the objects are separated by using Sobel's edge detector. Then, the heads are identified and projected onto an automatically generated ground plane. The interframe association uses the nearest neighbor principle.

In our setup, as depicted in Fig 1, the time-of-flight sensor is mounted 2.10m above ground, facing down. The sensor (Texas Instruments OPT8241) which we are using, is capable of generating 3D depth maps with a resolution of up to 320x240 pixel and a frame rate of up to 60Hz. The depth map thus generated reduces the problem of finding people's heads to finding local minimas in the map. An algorithm mimicking water flowing down and eventually collecting around the points of local minimas is described in [1]. Because of the depth map's noise character other local minimas finding algorithms (mean shift, random forest) are often not suitable. The water filling algorithm is claimed to be more robust and scale-invariant.

All objects are represented by a set of features, which are used to match new detected targets with tracks drawn by consecutive occurrences of persons already observed. A track's progress is followed and evaluated directly after reaching a zone opposite to where it started. The observable scene is divided into three zones (In, Out, Mid) with adjustable proportions.

II. TIME-OF-FLIGHT SENSOR

In computer vision, there is a variety of approaches for people counting and surveillance. Recent research has pointed out the beneficial influence of depth information in such systems, e.g. [1], [3]. In comparison to 2D color or monochrome image sensors, 3D depth sensors are not significantly affected by shading, changing illumination in the observed scene or variation of an object's color. By using depth information, the process of separating multiple objects, especially in crowded

environments, is not influenced by misrepresentation of spatial distances in 2D projections. Tasks such as background extraction can be accomplished using low-cost computational algorithms.

There is a variety of methods used to acquire 3D images. We considered two technologies for the people counting system:

A. Stereo Vision

With stereo vision, the depth information is generated via two spatially separated image sensors determining the disparity between both images [2].

B. Time-of-Flight

Time of flight systems use an actively modulated light source to illuminate the scene. By measuring reflected light and correlating it with modulated illumination, a phase shift can be observed and converted into depth information. In addition to the depth map, an intensity map is provided by the ToF system. The latter is used to gain measurement confidence for each pixel.

C. Comparison

Measuring the phase shift of illuminated and reflected light with ToF systems is subject to restrictions in terms of dynamic range with a restricted maximal measurable distance (also called ambiguity distance) depending on the illumination modulation frequency. Stereo vision systems, on the other hand, cannot handle great disparities occurring in short range. The computing effort on disparity information is relatively high, whereas ToF systems capture a direct measurement of the depth and thus, no costly processing is needed. By illuminating the scene with IR-spectrum light, ToF systems are robust in low light conditions.

TABLE I: Comparison

Compared characteristics	Stereo Vision	Time of Flight
Range	High	Adjustable
Complexity of depth extraction	High	Marginal
Sensor size	Medium	Compact
Low light performance	Low ¹	High
High ambient light performance	High	High

¹depending on sensor

The compact design and better runtime of the system led to the conclusion that the ToF sensor technology should be used. The depth map can be converted into a point cloud that can be displayed in a 3D-Viewer. Fig. 2 shows an example point cloud mapped with intensity values.

III. PEOPLE COUNTING ALGORITHM

The outline of our algorithm is shown in Fig. 3. The algorithm must fulfill the following requirements:

- We assume that we need an object to be visible for at least 5 frames
- FoV is in the reality 59° (taken 60° for simplicity)
- The length of the area covered by the ToF-Camera: $s = \frac{2 \cdot h}{\sqrt{3}} = 2.31m$

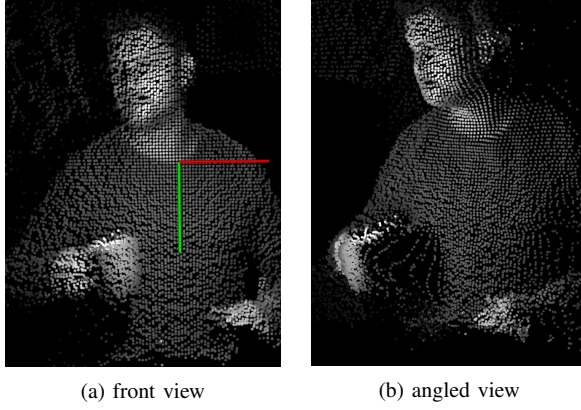


Fig. 2: Example point cloud mapped with intensity values: Person with cup

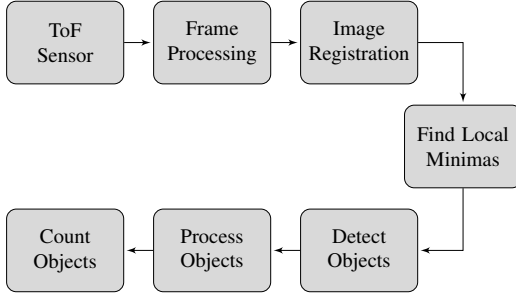


Fig. 3: Block diagram of our algorithm

- The velocity is assumed to be max. $12\text{km/h} \approx 3.5\text{m/s}$
- Using $t = \frac{s}{v}$, person moving with $v = 7\text{m/s}$ is visible for $t_7 = \frac{2.31\text{m}}{7\text{m/s}} = 0.33\text{s}$, the frame time has to be $0.33/5 = 0.066$ which means that we need to process $\approx 15.2\text{fps}$
- Accordingly, to count objects moving with $v = 3.5\text{m/s}$ we need to process at least ≈ 7.6 frames per second

After the frames are read, either directly from cameras (online operation) or from recorded files (offline), they undergo a preprocessing step. The depth (Fig. 4a) and the intensity (Fig. 4b) maps collected from the ToF camera are utilized to model the scene's background and to extract foreground objects (Fig. 4c), if present.

A. Preprocessing

The frame's data is converted into a data type supported by OpenCV. Then, the map is then denoised by simple morphological operations. Observing the mean value of the depth map (Fig. 5), one can determine whether there are any foreground objects in the field of view. If no objects are detected, the current frame is utilized to model the depth background. The background is averaged over the previous 50 background candidates and thus dynamically adjusted.

Dropping below a threshold, the mean depth value indicates that there are possibly foreground objects in the scene (the

threshold's value has to be estimated experimentally). To generate a foreground mask, the extracted depth foreground and a confidence information are used.

A more illustrative example, with all intermediate steps, is presented in (Fig. 6).

B. Waterfiller

Since it is expected that more than one person will be present in the scene, we have adopted the water filling algorithm [1] to find local minimas directly in the depth map generated by the ToF sensor. The algorithm works by analogy to the phenomenon known from nature. Rain drops falling down on some non-trivial-shaped surface will flow to local minimas, forming basins filled with water. These water containing regions allow to robustly distinguish between foreground objects.

The basic idea behind the water filling algorithm is depicted in (Fig. 7). The top picture represents the sensor's view of the scene. Although there are four persons in the field of view, for the sake of illustration, the side view shows only three of them. In the 2D representation one can notice concavities resembling human silhouettes turned upside down. The minimas reflect the shoulders and heads. After letting the uniformly distributed rain drops fall down, the shapes of water can be recovered. Eventually, only blobs of sufficient 'depth' are kept and tracked.

It is worth mentioning that the authors in [1] propose an improvement to the algorithm. The patch based water filling variant takes advantage of the fact that, in general, a liquid starting at some point (x, y) will flow to the same minimum, as the liquid starting in the neighborhood of the point (x, y) . Now, not only the 'size' of a water drop, but also the amount of water per raindrop can vary. As a consequence, as long as a concavity is found, it is instantly filled with more fluid than before.

Since the foreground regions have already been estimated, we can further optimize the algorithm by letting the flow start (uniformly distributed) only within the contours of detected persons. The third parameter controls the amount of water per foreground pixel, making the total volume dependent on the objects' dimensions.

After all raindrops have been allocated, an adaptive thresholding method is applied to recover the shapes found. In contrast to using a static threshold as suggested in [1], we adaptively change the threshold, due to the fact that the perspective of the sensor does not guarantee that the height remains the same. To deal with outliers, we set the threshold depending on the maximum plateau of the predefined size and the smallest non-zero value of the water map. The fourth parameter influencing the algorithm is the `thresholdDivider`. It impacts the outcome according to the following formula:

$$threshold = w_{max} - \frac{w_{max} - w_{min}}{thresholdDivider}$$

This set of four parameters has to be determined experimentally, taking into account the camera position, the distance to objects and the time of computation.

C. Object Detection

The next step is the detection of objects based on the output of the water filling algorithm.

Each contour with an area larger than a particular threshold is considered to be a new-found object and gets annotated with a few attributes: a list of all pixels forming this object, its bounding box, the center of mass (3D) and the zone in which it is currently present.

D. Object Processing

Since detection is not enough, a method for interframe association of objects needs to be implemented. Here, in our test-based matching algorithm, we use the object's properties to analyze if the new-found target can be paired with any others being already watched. In the case of a match, the target being tracked gets updated with the new data. Otherwise, as long as the object did not appear in the middle zone, a new target is created.

Each tracked person has its own Kalman filter to estimate their position and the dimensions of bounding box in the next frame. Based on the object's data and the Kalman predictions, a set of tests can be designed. An important feature is that the collection can be easily extended to better solve complicated cases or to make use of objects' properties that can be introduced in the future (i.e. by thermal and color cameras).

For the time being, it was sufficient to run just four tests.

Nearest Neighbor: The distances between the center of the new detected object and the centers of all targets that were being already followed (or their predictions) are calculated and the one with smallest distance is chosen.

Bounding Box: If a new object appears inside an already tracked bounding box (or its prediction), there is a match.

Detection of a merge: If two targets come so close, that the waterfiller cannot separate them, we make use of the Kalman filter predictions. It is likely a merge if predicted centers of two followed targets are inside the new-found object's bounding box and the sum of areas of the predicted bounding boxes equals approximately the area of the new-found one.

Detection of a split: In the opposite situation, one larger object splits into a few smaller pieces. This situation can be picked out if, according to the Kalman prediction, a single large bounding box is expected, but only few smaller ones can be found.

The results of all tests are represented by binary matrices. Matches are represented by 1 on the intersections between the new object (rows), and the old ones (columns).

The matrices are then interpreted by a rule-based algorithm.

E. Object Counting

All tracks reaching the zone opposite to where they started are counted immediately. This prevents the 'u-turn' effect from occurring. When one person exits and another enters the field of view at the same time, the tracker could misinterpret it as an object turning back. As a consequence, after exiting the frame, the counter would register that the observed person

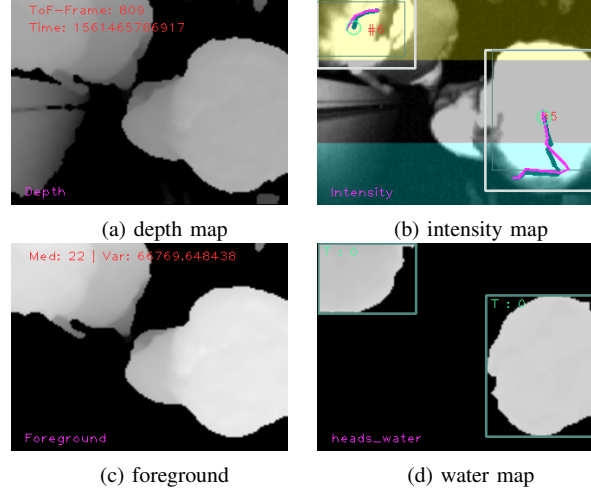


Fig. 4: The figure shows the depth and the intensity map (counting zones applied) obtained from the ToF camera, the estimated foreground and the water map generated by the water filling algorithm

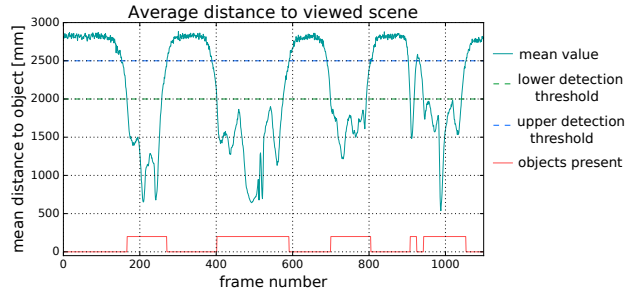


Fig. 5: Pre-detection of objects based on mean distance to the viewed scene

entered and left the scene in the same zone, thus they would not be counted.

IV. DISCUSSION

The above application was implemented using C++ and an open-source OPENCV library, without any platform specific optimizations, e.g. ISSE vector extensions. The testing platform consisted of an Intel Core i7 (5th Generation) equipped with 16 GB of RAM.

While testing the algorithm we observed a couple of problems which degraded the counting accuracy and the runtime.

A. Runtime

Trying to optimize our algorithm in terms of frames processed per second we discovered that the most time-consuming part was the water filling algorithm. To evaluate the runtime behavior of the water filling algorithm itself, we profiled the software accordingly. The corresponding result as given in (Fig. 8) shows that the amount of processing time required

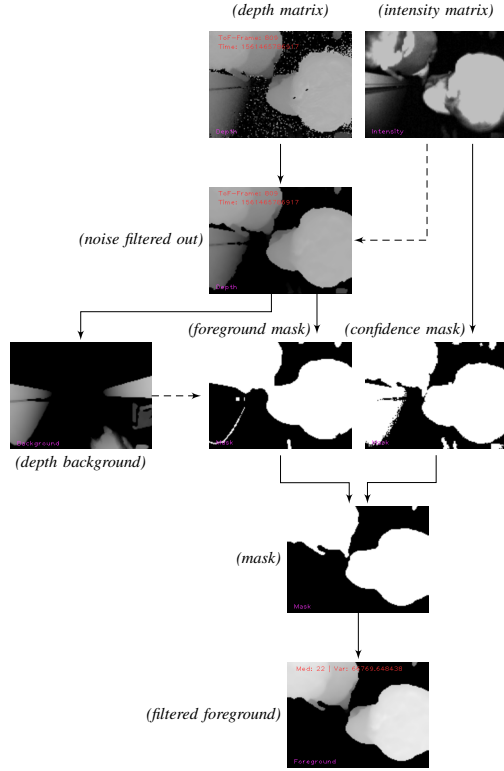


Fig. 6: Foreground extraction algorithm

grows quadratically with growing resolution. Due to its strictly iterative nature, there is no simple method to parallelize it. An appealing idea was to calculate the water map on down-scaled data first, then to resize it to our target resolution and, finally to refine it. It turned out that, since we were not aiming for pixel-wise accuracy, no enhancement was needed.

TABLE II: Computation time of the water filling algorithm for different resolutions

Resolution [px]	Frames per second
160x120	36.6
80x60	60
40x30	70

The table II shows an increase of over 60% by just quartering the original resolution. Surprisingly, the counting accuracy was barely affected, even when the depth map was scaled down to a 40x30 pixel image. Due to that, we typically run the water filling algorithm with the resolution of 80x60 pixel without influencing the counting accuracy.

B. Motion artifacts

ToF sensors always produce a certain amount of noise. Even though most sources of noise can be eliminated by filtering, motion artifacts can substantially interfere with the water filling algorithm. Objects passing the sensor in close proximity ($< 20\text{cm}$) and thus possessing a high relative velocity are

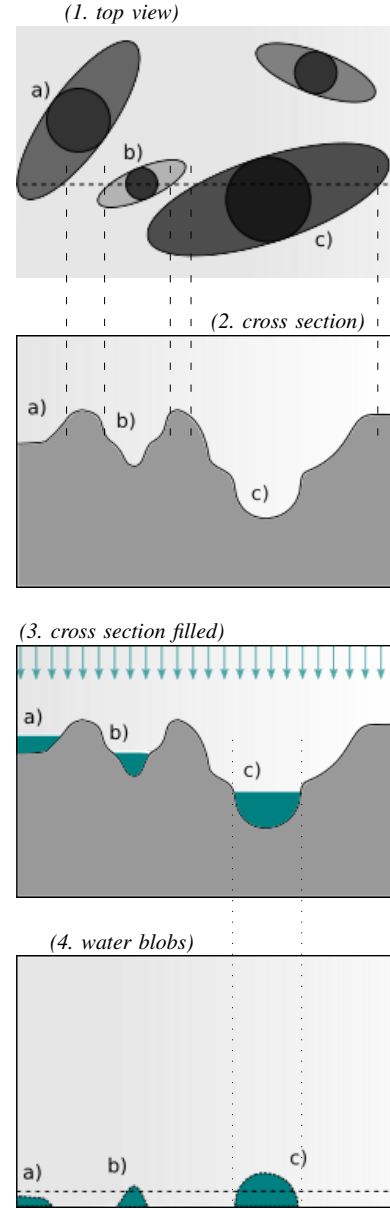


Fig. 7: Waterfiller: top view with marked cross section, shape of the cross section, cross section filled with water, water blobs

causing significant outliers that modify the object's shape. These could not be filtered effectively. Grave fluctuations in originally homogeneous areas lead to a poorly set threshold resulting in porous water maps complicating and potentially averting object detection. To counteract this issue the threshold does not depend on the water maps maxima but the highest observed area of a pre-defined size. To minimize this effect and improve robustness, object tracks without subsequent update objects are maintained and updated with predictions for a limited amount of frames.

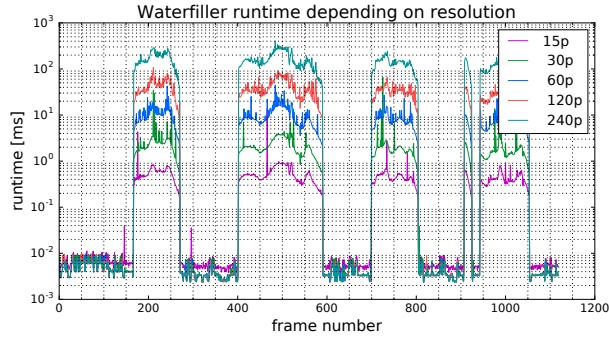


Fig. 8: Runtime of a waterfiller algorithm for different resolution of input matrices

V. RESULTS

In order to evaluate and test the developed algorithms we created a dataset consisting of three different test situations:

- People walking in the same direction with some clearance
- People walking in opposing directions with occasional contact
- A highly crowded transit area with large groups of people

The test situations were designed to challenge the algorithms with special cases such as unusual body posture (e.g. people drinking out of bottles while passing through the transit area) or colliding with other people. The test scenarios were captured with indoor daylight conditions and without direct light irradiation. The limited observed space allows a maximum of 3-4 people in the frame.

Our results (Table III) summarize the counting accuracy of the system with variation of the water filling resolution. Incorrect counting was caused by strong motion artifacts and resulted in erroneous tracking. The results indicate a low dependency of the waterfilling algorithm on the resolution used. A further reduction of the resolution is not notably beneficial to the system's runtime and therefore not taken into account. A high counting accuracy is achieved regardless of the number of people in the scene.

TABLE III: Experimental results

Waterfiller resolution	True in	True out	In cc/fp/nc	Out cc/fp/nc ¹	Accuracy %
160x120	82	84	82/0/0	83/0/1	99.40
80x60	82	84	81/0/1	84/0/0	99.40
40x30	82	84	82/0/0	82/0/2	98.80

¹cc: correctly counted, fp: false positive, nc: not counted

VI. UPCOMING WORK

Based on the dataset we have generated it has been shown that the counting accuracy is quite high. During further testing we expect to find situations where the presented algorithms could fail if new scenes are used. For this reason we will work on a real life installation of a complete smart camera based

on our algorithms using NXP's i.mx6 System on Module series to generate more test data. Based on a more comprehensive dataset we will work on a deep learning solution to compare the presented method against machine learning concepts especially with respect to computational complexity. To further extend the functionality of the application, we're working on the integration of other sensors such as thermal and color cameras. Our goal is to differentiate vital from non-vital objects in the scene.

VII. CONCLUSIONS

In this paper we presented a comprehensive application for counting people using public transportation based on 3D data generated by a ToF camera. Apart from the preprocessing and tracking algorithms of the application, the core algorithm used to analyze the 3D scene is based on the water filling algorithm. Contrary to our expectations, the waterfiller can be used at very low resolutions, and still provides reliable results. While applying enhanced methods for postprocessing of the 3D data and tracking, we could notably improve the counting accuracy by dynamically adapting the threshold. Applying our techniques, we could show that we could notably reduce the distance between the sensor and the scene, compared to other work in this field. The application has been implemented with embedded systems in mind, where low computational complexity plays an important role. For this reason, we have enhanced not only the signal processing itself, but the runtime footprint as well.

REFERENCES

- [1] Zhang, Xucong & Yan, Junjie & Feng, Shikun & Lei, Zhen & Yi, Dong & Li, Stan. "Water Filling: Unsupervised People Counting via Vertical Kinect Sensor," Proceedings - 2012 IEEE 9th International Conference on Advanced Video and Signal-Based Surveillance, AVSS 2012. 215-220. 10.1109/AVSS.2012.82.
- [2] M. Werner, B. Stabernack and C. Riechert, "Hardware implementation of a full HD real-time disparity estimation algorithm," in IEEE Transactions on Consumer Electronics, vol. 60, no. 1, pp. 66-73, February 2014.
- [3] N. Bernini, L. Bombini, M. Buzzoni, P. Cerri and P. Grisleri, "An embedded system for counting passengers in public transportation vehicles," 2014 IEEE/ASME 10th International Conference on Mechatronic and Embedded Systems and Applications (MESA), Senigallia, 2014, pp. 1-6.
- [4] S. Mukherjee, B. Saha, I. Jamal, R. Leclerc and N. Ray, "Anovel framework for automatic passenger counting," 2011 18th IEEE International Conference on Image Processing, Brussels, 2011, pp. 2969-2972.
- [5] X. Jianzhong, Z. Qiuyu, Y. Sai and S. Wenjun, "Passenger counting based on Kinect," 2014 International Conference on Audio, Language and Image Processing, Shanghai, 2014, pp. 405-409.
- [6] A. Bevilacqua, L. D. Stefano and P. Azzari, "People Tracking Using a Time-of-Flight Depth Sensor," 2006 IEEE International Conference on Video and Signal Based Surveillance, Sydney, Australia, 2006, pp. 89-89.
- [7] E. Bondi, L. Seidenari, A. D. Bagdanov and A. Del Bimbo, "Real-time people counting from depth imagery of crowded environments," 2014 11th IEEE International Conference on Advanced Video and Signal Based Surveillance (AVSS), Seoul, 2014, pp. 337-342.
- [8] P. Bamrungrathai and S. Puengsawad, "Robust people counting using a region-based approach for a monocular vision system," 2015 International Conference on Science and Technology (TICST), Pathum Thani, 2015, pp. 309-312.